

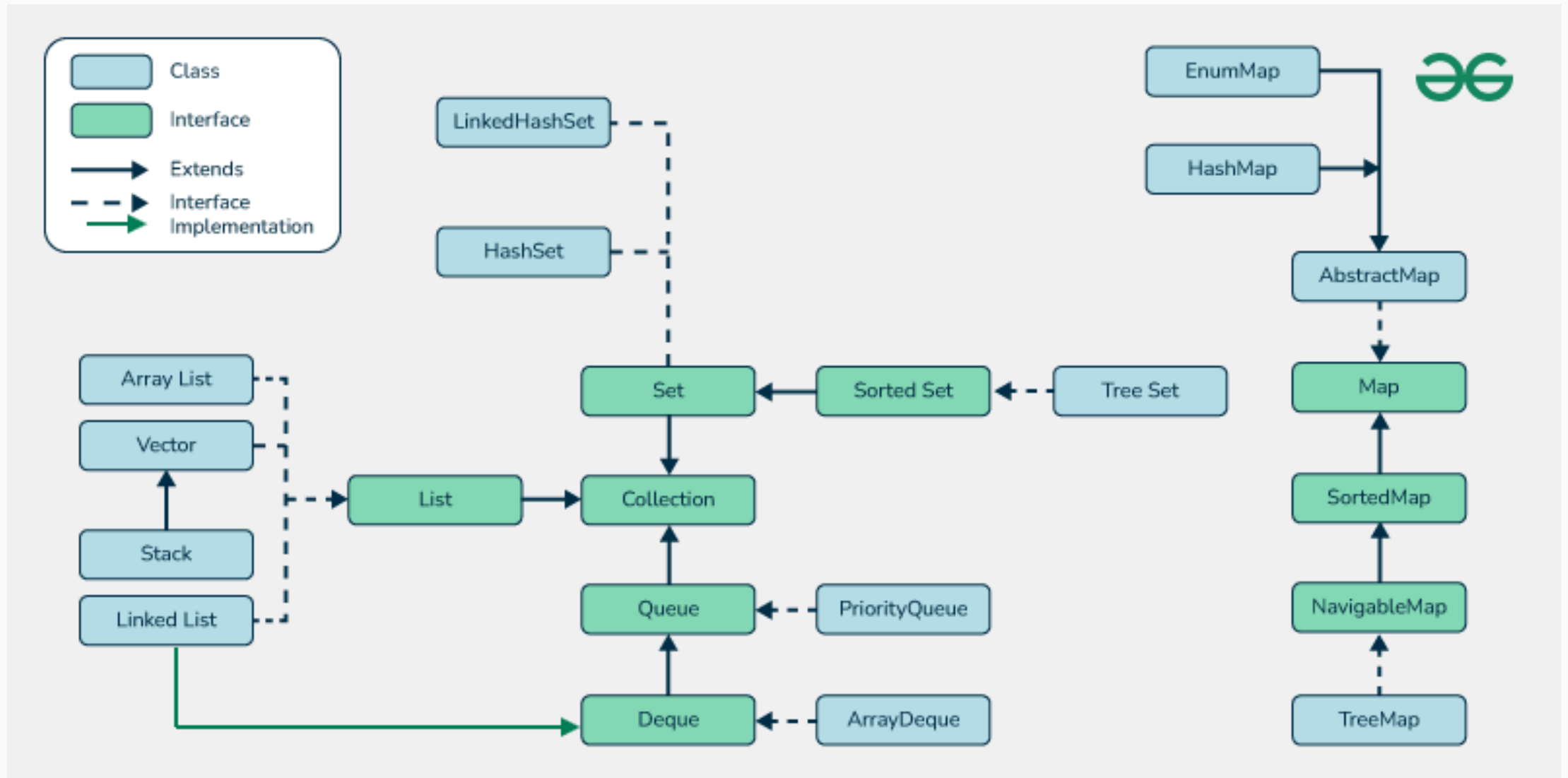
Java Advanced Schulung 2026

Lea Föcke & Phil Steinhorst



Rückblick: Tag 1

- Wiederholung von Grundbegriffen
- Dokumentation von Code
- Generics
- Interfaces
- Datenstrukturen



<https://www.geeksforgeeks.org/collections-in-java-2/>

Erzeugung von Datenstrukturen

- Gestern gesehen: Um "mal eben" eine Liste zu erzeugen, sind viele Schritte notwendig:

```
List<Integer> ls = new LinkedList<>();  
ls.add(4);  
ls.add(8);  
ls.add(15);  
ls.add(16);
```

- Ein Ausweg: Nutzung von `Arrays.asList(...)` :

```
List<Integer> ls = Arrays.asList(4, 8, 15, 16, 23, 42);
```

- Dies ist jedoch ein Umweg, bei dem zuerst ein Array erzeugt wird.

Fabrikmethode für Listen

Schneller geht's mit seit Java 9 existierenden Fabrikmethoden:

```
List<Integer> ls = List.of(4, 8, 15, 16, 23, 42);
```

Die erzeugte Liste...

- ...ist nur lesbar.
- ...kann nicht `null` enthalten.
- Kann identisch zu einer Liste mit denselben Elementen sein, muss aber nicht:

```
List<Integer> x = List.of(4, 8, 15, 16, 23, 42);  
List<Integer> y = List.of(4, 8, 15, 16, 23, 42);  
x == y // kann true sein, muss aber nicht!
```

Fabrikmethode für Sets und Maps

Analog für `Set` :

```
Set<Integer> s = Set.of(4, 8, 15, 16, 23, 42);
```

- Das erzeugte Set darf zusätzlich keine Duplikate enthalten

Und für `Map` :

```
Map<Integer, String> m = Map.of(1, "Schnitzel", 3, "Rösti", 4, "Bowl");  
m.get(3); // Rückgabe: "Dönertasche"
```

- Auch hier: Keine Keys dürfen doppelt vorkommen.

Aufgabe 1

Siehe `Tag2_1_Mehr_Collections.pdf` .

Vergleichen von Werten

- Zahlenwerte können in Java einfach mittels `<`, `>`, `==`, ... verglichen werden.
- Für Referenzdatentypen (außer Zahlen) geht das nicht.
- **Beispiel:** Sortieren eines Arrays von Strings:

```
String[] a = {"Lava", "Leben", "leben", "Licht"};  
Arrays.sort(a);  
System.out.println(a[3]); // Was ist die Ausgabe?
```


Lexikographische Sortierung

- Strings werden **lexikographisch** sortiert.
- **Vorsicht:** *lexikographisch* $\neq$ *wie im Lexikon!*
- Verglichen wird zeichenweise anhand des Unicode-Codepunktes. Groß- und Kleinbuchstaben sind daher **nicht** gleichwertig.

```
String[] a = {"Lava", "Leben", "leben", "Licht"};  
Arrays.sort(a); // jetzt ist a = {"Lava", "Leben", "Licht", "leben"}
```

- **Frage:** Woher weiß `Arrays.sort(...)`, dass Strings so sortiert werden?

Arrays.sort

- **Antwort:** `Arrays.sort(...)` weiß das nicht.

sort

```
public static void sort(Object[] a)
```

Sorts the specified array of objects into ascending order, according to the natural ordering of its elements. All elements in the array must implement the `Comparable` interface. Furthermore, all elements in the array must be *mutually comparable* (that is, `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the array).

- Die Methode nutzt das `Comparable` -Interface, welches die Sortierung definiert.
- Dieses besteht einer einzigen Methode `compareTo` , die zum Vergleich von Werten eines Referenzdatentyps benutzt wird.

String.compareTo(...)

- Die Klasse `String` implementiert das `Comparable` -Interface:

`compareTo`

```
public int compareTo(String anotherString)
```

Compares two strings lexicographically. The comparison is based on the Unicode value of each character in the strings. The character sequence represented by this `String` object is compared lexicographically to the character sequence represented by the argument string. The result is a negative integer if this `String` object lexicographically precedes the argument string. The result is a positive integer if this `String` object lexicographically follows the argument string. The result is zero if the strings are equal; `compareTo` returns 0 exactly when the `equals(Object)` method would return true.

Comparable-Interface

```
int compareTo(T o)
```

Compares this object with the specified object for order. Returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object.

The implementor must ensure `signum(x.compareTo(y)) == -signum(y.compareTo(x))` for all `x` and `y`. (This implies that `x.compareTo(y)` must throw an exception if and only if `y.compareTo(x)` throws an exception.)

The implementor must also ensure that the relation is transitive: `(x.compareTo(y) > 0 && y.compareTo(z) > 0)` implies `x.compareTo(z) > 0`.

Finally, the implementor must ensure that `x.compareTo(y)==0` implies that `signum(x.compareTo(z)) == signum(y.compareTo(z))`, for all `z`.

API Note:

It is strongly recommended, but *not* strictly required that `(x.compareTo(y)==0) == (x.equals(y))`. Generally speaking, any class that implements the `Comparable` interface and violates this condition should clearly indicate this fact. The recommended language is "Note: this class has a natural ordering that is inconsistent with equals."

Anforderungen an `int compareTo(T o)`

- `x.compareTo(y)` ist negativ, wenn `x` "kleiner" als `y`.
- `x.compareTo(y)` ist positiv, wenn `x` "größer" als `y`.
- `x.compareTo(y)` ist 0, wenn `x` und `y` "gleich groß".
- `x.compareTo(y)` ist negativ genau dann, wenn `y.compareTo(x)` positiv.
- Wenn `x.compareTo(y)` und `y.compareTo(z)` positiv sind, dann auch `x.compareTo(z)`.
- Wenn `x.compareTo(y)` 0 ist, dann haben `x.compareTo(z)` und `y.compareTo(z)` immer dasselbe Vorzeichen.
- → dient als Grundlage für Testfälle.

Comparable

- Mit `compareTo` kann die **natürliche Ordnung** von Objekten einer Klasse definiert werden, die zum Beispiel zum Sortieren verwendet wird.
- Meist benutzt man dafür Attribute, die ebenfalls `Comparable` sind.
- Daher bietet es sich oft an, auf die `compareTo`-Methoden der Attribute zurückzugreifen.

```
@Override
public int compareTo(Tier o) {
    // vergleiche erst nach Größe, dann nach Name
    int vergleichGroesse = Double.valueOf(this.groesse).compareTo(o.groesse);
    if (vergleichGroesse != 0)
        return vergleichGroesse;
    return this.name.compareTo(o.name);
}
```


Nachteile

- Nicht immer ist die natürliche Ordnung für jede Situation die gewünschte.
- **Problem:** `Comparable` erlaubt nur das Definieren *einer* Ordnung.
- Um die natürliche Ordnung zu umgehen, kann man in einer eigenen Klasse einen `Comparator` implementieren.
- Dies bietet sich vor allem an, wenn man die eigentliche Klasse nicht ändern will (oder kann).
- → `Tag_2_2_Comparator.pdf`

Comparator

- Erfordert Implementierung der Methode `compare(T x, T y)`, welche zwei Objekte einer Klasse `T` vergleicht.
- Die Semantik ist völlig analog zu `compareTo`:

```
int compare(T o1,  
           T o2)
```

Compares its two arguments for order. Returns a negative integer, zero, or a positive integer as the first argument is less than, equal to, or greater than the second.

The implementor must ensure that `signum(compare(x, y)) == -signum(compare(y, x))` for all `x` and `y`. (This implies that `compare(x, y)` must throw an exception if and only if `compare(y, x)` throws an exception.)

The implementor must also ensure that the relation is transitive: `((compare(x, y)>0) && (compare(y, z)>0))` implies `compare(x, z)>0`.

Finally, the implementor must ensure that `compare(x, y)==0` implies that `signum(compare(x, z))==signum(compare(y, z))` for all `z`.

Comparator: Verwendung

- Einige `Collections` -Methoden sind überladen, sodass man ihnen optional einen `Comparator` mitgeben kann:
 - `binarySearch`, `min`, `max`, `sort`
- Wenn der enthaltene Datentyp kein `Comparable` ist, **muss** man dies sogar.
- Bei Verwendung erzeugt man üblicherweise *ad hoc* einen `Comparator` beim Methodenaufruf, wenn man diesen nicht mehrfach benutzt:

```
LinkedList<Tier> ls = ... ;  
Collections.sort(ls, new MeinTierComparator());
```

Comparator bei impliziter Sortierung

- Bei Collections, die ihre Elemente implizit sortieren, kann man den `Comparator` bei Erzeugung übergeben:

```
TreeSet<Tier> ts = new TreeSet(new MeinTierComparator());
```

- Auch hier: Wenn der enthaltene Datentyp kein `Comparable` ist, **muss** man dies sogar.